

11. Generalized Regression Neural Network (GRNN)

Algorithm implemented: Specht's GRNN (Nadaraya-Watson kernel regression) from scratch.

Dataset: Noisy 1D multi-modal benchmark function.

```
# ---- Core Libraries ----
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.base import BaseEstimator, RegressorMixin
from sklearn.model_selection import KFold, train_test_split
from sklearn.metrics import r2_score, root_mean_squared_error

# Styling setup
plt.style.use('seaborn-v0_8-whitegrid' if 'seaborn-v0_8-whitegrid' in plt.style.available else
'default')
plt.rcParams['font.sans-serif'] = 'DejaVu Sans'
plt.rcParams['figure.figsize'] = (10, 6)
plt.rcParams['figure.dpi'] = 110
np.random.seed(42)
```

GRNN Architecture

Step-by-Step Algorithm (Generalized Regression Neural Network)

1. **Input layer:** receive the d-dimensional query vector x and pass it to every pattern neuron.
2. **Pattern layer:** with N training points, each unit i computes a Gaussian activation $p_i(x) = \exp(-||x - x_i||^2 / (2\sigma^2))$.
3. **Summation layer:** compute two sums over all pattern units:
 - $S(x) = \sum_i y_i \cdot p_i(x)$ (weighted by training targets)
 - $D(x) = \sum_i p_i(x)$ (normalization)
4. **Output layer:** predict the conditional mean $\hat{y}(x) = S(x) / D(x)$ (Nadaraya-Watson estimator).
5. **Training:** one pass - just store the training set; **no** backpropagation or iterative optimization.
6. **Tuning:** only σ must be chosen (small σ = memorizes noise; large σ = over-smooth).

```
class GRNN(BaseEstimator, RegressorMixin):
    """
    Generalized Regression Neural Network (Specht, 1991) - vectorized implementation.

    Prediction is the Nadaraya-Watson kernel regression estimate:
        y_hat(x) = sum_i y_i * exp(-||x - x_i||^2 / (2*sigma^2))
                  / sum_i      exp(-||x - x_i||^2 / (2*sigma^2))

    Training is one-pass: the model simply stores the training set (no gradient descent).
    The only hyperparameter is the smoothing spread sigma.
    """

    def __init__(self, sigma=1.0):
        self.sigma = sigma

    def fit(self, X, y):
        # "Training" = memorizing the data (pattern + summation layers)
        self.X_train_ = np.asarray(X, dtype=np.float64)
        self.y_train_ = np.asarray(y, dtype=np.float64).ravel()
        return self
```

```
def predict(self, X):
    X_test = np.asarray(X, dtype=np.float64)

    # ---- Pattern Layer: squared Euclidean distance between each test and train point ----
    # Uses the expansion  $\|a-b\|^2 = \|a\|^2 + \|b\|^2 - 2 a \cdot b$  for efficiency.
    dist_sq = np.sum(X_test**2, axis=1, keepdims=True) + \
        np.sum(self.X_train**2, axis=1, keepdims=True).T - \
        2 * np.dot(X_test, self.X_train.T)
    dist_sq = np.maximum(dist_sq, 0.0) # guard against tiny negative values from rounding

    # ---- Pattern Layer activation: Gaussian kernel of each training point ----
    kernels = np.exp(-dist_sq / (2.0 * (self.sigma ** 2)))

    # ---- Summation Layer: D-sum (denominator) and S-sum (numerator) ----
    D = np.sum(kernels, axis=1) # unweighted sum of activations
    S = np.dot(kernels, self.y_train_) # activation-weighted sum of targets

    # ---- Output Layer: normalized weighted average (prevents 0/0 for isolated queries) ----
    D_safe = np.where(D < 1e-12, 1e-12, D)
    return S / D_safe
```

```
print("Custom vectorized GRNN class defined!")
```

Custom vectorized GRNN class defined!

Smoothing Parameter σ

The Role of the Smoothing Parameter σ

1. Fit GRNNs with a small, medium and large σ on the same noisy 1D data.
2. Small σ (0.05): each training point dominates its own neighborhood -> wiggly curve, high variance.
3. Large σ (1.50): all points influence every query -> flat curve, high bias.
4. Medium σ (≈ 0.35): smooth interpolation that follows the true underlying function.
5. Compare training R^2 and curve shapes to see the bias-variance trade-off.

Hyperparameter Optimization Process (5-Fold CV)

1. Define a grid of candidate σ values from 0.05 to 1.0.
2. Split the data into 5 fixed folds.
3. For each σ : fit on 4 folds, compute RMSE on the held-out fold, then average the 5 scores.
4. Select the σ with the lowest average CV RMSE.
5. Plot the CV error curve to confirm a clear optimum.

```
# ---- Synthetic 1D regression problem ----
np.random.seed(42)
X_syn = np.sort(np.random.uniform(-3, 3, 100)).reshape(-1, 1)
# True Latent function:  $\sin(2x) + \cos(0.5x^2)$ ; observed targets add Gaussian noise
y_true_curve = lambda x: np.sin(2 * x) + np.cos(0.5 * x**2)
y_syn = y_true_curve(X_syn).ravel() + np.random.normal(0, 0.18, 100)

X_eval = np.linspace(-3.2, 3.2, 300).reshape(-1, 1) # dense grid for plotting

# Under-smoothing, near-optimal, over-smoothing
sigmas = [0.05, 0.35, 1.50]
colors = ['purple', 'crimson', 'teal']

fig, axes = plt.subplots(1, 3, figsize=(18, 5))

for ax, sig, col in zip(axes, sigmas, colors):
    # Fit a fresh GRNN for each sigma (training is instant)
    grnn_demo = GRNN(sigma=sig).fit(X_syn, y_syn)
    y_pred_demo = grnn_demo.predict(X_eval)
```

```

ax.scatter(X_syn, y_syn, color='navy', s=35, alpha=0.7, label='Training Data')
ax.plot(X_eval, y_true_curve(X_eval), 'k--', lw=1.5, label='True Signal')
ax.plot(X_eval, y_pred_demo, color=col, lw=2.5, label=rf'GRNN ( $\sigma$ ={sig})')

# Training R^2 shows the bias-variance behavior for each sigma
r2 = r2_score(y_syn, grnn_demo.predict(X_syn))
ax.set_title(rf"Smoothing  $\sigma$  = {sig} (Train  $R^2$  = {r2:.3f})", fontweight='bold')
ax.set_xlabel("x")
ax.set_ylabel("y")
ax.legend(loc='lower left')

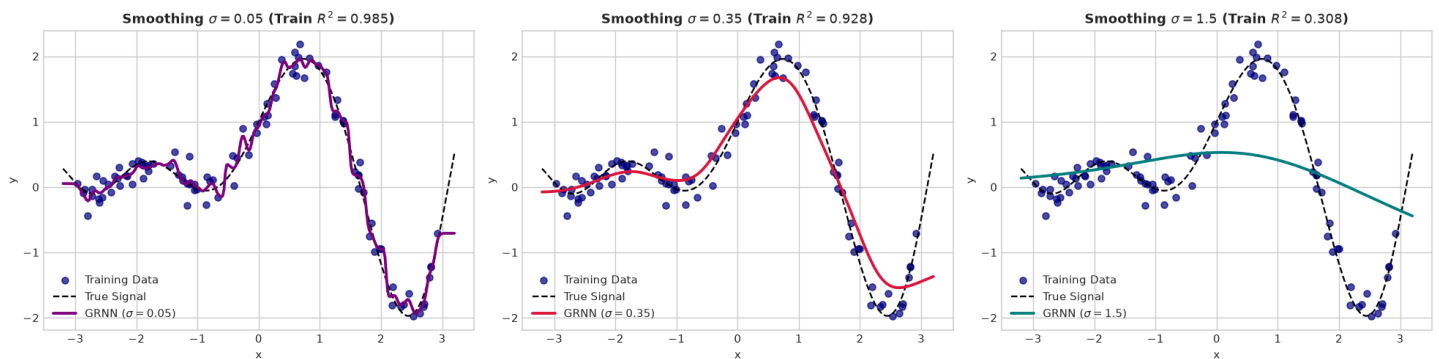
```

```

plt.suptitle("GRNN Behavior Under Different Smoothing Spread Values ( $\sigma$ )", fontsize=15,
fontweight='bold')
plt.tight_layout()
plt.show()

```

GRNN Behavior Under Different Smoothing Spread Values (σ)



```

# ---- 5-fold CV grid search for the optimal smoothing sigma ----
sigma_candidates = np.linspace(0.05, 1.0, 50)
cv_scores = []
kf = KFold(n_splits=5, shuffle=True, random_state=42) # fixed folds for a fair comparison

```

```

for sig in sigma_candidates:
    fold_rmse = []
    for train_idx, val_idx in kf.split(X_syn):
        # Split the 1D data into this fold's train/validation parts
        X_tr, y_tr = X_syn[train_idx], y_syn[train_idx]
        X_va, y_va = X_syn[val_idx], y_syn[val_idx]

        # Fit GRNN on the training fold and score on the validation fold
        model = GRNN(sigma=sig).fit(X_tr, y_tr)
        preds = model.predict(X_va)
        fold_rmse.append(root_mean_squared_error(y_va, preds))

```

```

cv_scores.append(np.mean(fold_rmse)) # average CV error for this sigma

```

```

best_idx = np.argmin(cv_scores)
best_sigma = sigma_candidates[best_idx]
print(f"Optimal Smoothing Parameter  $\sigma$ : {best_sigma:.3f} (CV RMSE: {cv_scores[best_idx]:.4f})")

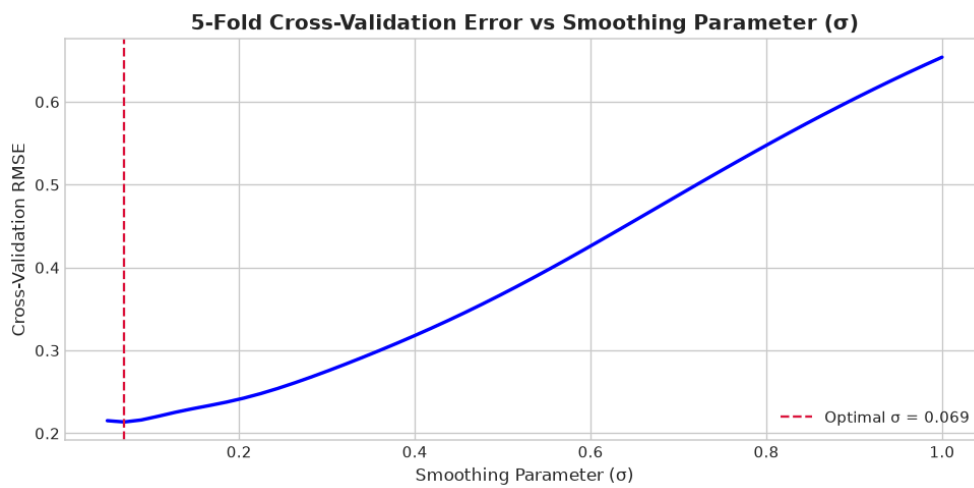
```

```

# ---- Plot the CV error curve and mark the optimum ----
plt.figure(figsize=(9, 4.5))
plt.plot(sigma_candidates, cv_scores, 'b--', lw=2.2)
plt.axvline(x=best_sigma, color='crimson', linestyle='--', label=f'Optimal  $\sigma$  = {best_sigma:.3f}')
plt.title("5-Fold Cross-Validation Error vs Smoothing Parameter ( $\sigma$ )", fontsize=13, fontweight='bold')
plt.xlabel("Smoothing Parameter ( $\sigma$ )", fontsize=11)
plt.ylabel("Cross-Validation RMSE", fontsize=11)
plt.legend()
plt.tight_layout()
plt.show()

```

Optimal Smoothing Parameter σ : 0.069 (CV RMSE: 0.2137)



```
# ---- Final model: refit with the CV-selected sigma and evaluate on a held-out split ----
X_tr, X_te, y_tr, y_te = train_test_split(X_syn, y_syn, test_size=0.25, random_state=42)
final_model = GRNN(sigma=best_sigma).fit(X_tr, y_tr)
y_hat = final_model.predict(X_te)

print(f"Final GRNN with CV-selected sigma = {best_sigma:.3f}")
print(f" - Train R²: {r2_score(y_tr, final_model.predict(X_tr)):.4f}")
print(f" - Test R²: {r2_score(y_te, y_hat):.4f}")
print(f" - Test RMSE: {root_mean_squared_error(y_te, y_hat):.4f}")
```

```
Final GRNN with CV-selected sigma = 0.069
- Train R²: 0.9829
- Test R²: 0.9473
- Test RMSE: 0.1718
```